Abstract Factory: Payment Provider Integration

1. Roles that checkout needs

Looking at the starter code, CheckoutService only uses three things from a payment provider: it authorizes a payment, it refunds a payment and it verifies a webhook signature. We turned each of these into an abstract class so checkout can depend on them instead of on Stripe or Adyen classes.

Capability	Contract	Method
Authorization	PaymentAuthorizer	authorize(amount)
Refunds	RefundProcessor	refund(amount)
Webhook verification	WebhookVerifier	verify(signature)

2. Pattern and design

We used Abstract Factory. PaymentProviderFactory is an abstract class with three methods: create_authorizer, create_refund_processor and create_webhook_verifier. Each provider has its own concrete factory that returns its three classes.

Factory	Products it creates
StripePaymentFactory	StripeAuthorizer, StripeRefundProcessor, StripeWebhookVerifier
AdyenPaymentFactory	AdyenAuthorizer, AdyenRefundProcessor, AdyenWebhookVerifier
MercadoPagoPaymentFactory	MercadoPagoAuthorizer, MercadoPagoRefundProcessor, MercadoPagoWebhookVerifier

CheckoutService now receives a PaymentProviderFactory in its constructor and asks it for the three objects it needs. It no longer imports any provider class and it no longer has the if/elif on the provider name. The decision of which provider to use moved to PaymentApp, which works as the composition root. PaymentApp uses a PaymentProviderSelector that maps the configured name ("stripe", "adyen" or "mercado_pago") to the right factory and then passes that factory to checkout.

3. What varies?

The provider. Each provider has its own way to authorize, refund and verify webhooks, so the three concrete classes change from one provider to another, and so does the factory that creates them.

4. What stays stable?

The checkout logic and the contracts. Checkout always does the same three operations no matter which provider is behind it, and the four abstract classes did not change when we added Mercado Pago.

5. Why is this a family-level creation problem?

Because the three objects have to come from the same provider. It would not make sense to authorize with Stripe and then verify the webhook with Adyen's rules, since the payment, the refund and the webhook all belong to the same provider account. The original code already created the three objects together inside each branch of the if/elif. So the problem is not creating one object, it is creating a set of related objects that have to match.

6. Which pattern did we choose and why?

Abstract Factory. We considered Factory Method, but it fits better when only one product changes. Here three products change together, and Abstract Factory groups their creation in one object per provider. If checkout gets the Stripe factory, everything it creates is Stripe, so mixing providers is not possible. Switching to another provider means passing a different factory, nothing else.

7. Who selects the concrete provider family?

PaymentApp, together with PaymentProviderSelector, based on configuration. Checkout does not choose the provider, it only uses the factory it receives.

8. Adding Mercado Pago

To add Mercado Pago we created MercadoPagoAuthorizer, MercadoPagoRefundProcessor, MercadoPagoWebhookVerifier and MercadoPagoPaymentFactory, and registered the factory in PaymentApp. The file checkout_service.py was not modified.

9. What complexity cost does the design introduce?

The main cost is the number of classes. The starter had 7 classes and our version has 19: three product contracts, one factory contract, four classes per provider, plus checkout, the selector and the app. Following a call from checkout to the actual provider code also takes more steps than before.

Another cost is that adding a new capability is expensive. If checkout later needs something like payouts, we would have to add a method to PaymentProviderFactory and implement it in every concrete factory. Abstract Factory makes it easy to add new providers, but harder to add new products. With only two simple providers this structure could be too much, but it makes more sense as more providers are added.